

Computer Graphics

Instructor: MSc. Thai Trung Tin

Email: tttin@hcmiu.edu.vn

Lab 1 - Introduction to Computer Graphics

Part 1: In-Class Practices and Exercises

CG Lab 1 - Questions and Hints (Python + OpenGL)

Materials: [Lab 1](#)

Framework – window_glut.py

This file provides the GLUT 2D framework (Simple2DApp) that you will use throughout the lab.

It sets up gluOrtho2D(0, W, 0, H), handles callbacks, and includes helper functions for drawing (line, rectangle, oval, polyline, text, etc.).

Task:

For this exercise, do not re-implement the framework. Instead, use the provided functions in window_glut.py as your reference. Your job is to complete the exercise files (e.g., ex1_1_fan.py, ex4_star_complete.py) by calling these helpers.

Hints:

Look at how shapes and text are drawn in window_glut.py.

Functions like draw_line, draw_rect, draw_ellipse_outline, and draw_text are there to guide you.

Study their implementation style and reuse them when building your solutions.

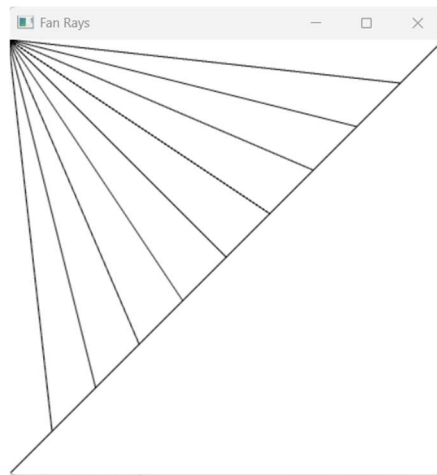
Ex 1A - Fan of Rays

Task:

In this exercise, you will implement the *draw* function inside the file *ex1_1_fan.py*.

To complete this task, carefully refer to the implementation of the *draw_line* function in *window_glut.py*.

From top-left (0, H), draw N rays to diagonal points ($t*W$, $t*H$) with $t=i/(N+1)$. Draw the diagonal too.



Hints:

Recompute width/height after resize; loop i in $1..N$; thin line width.

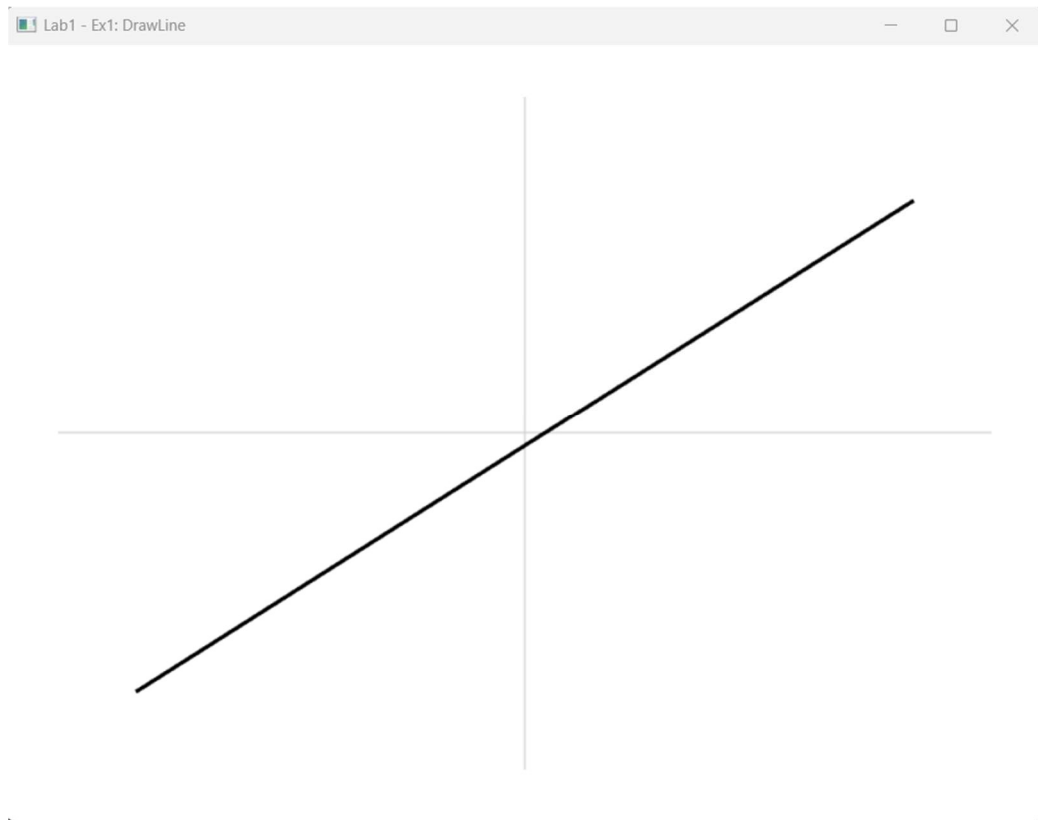
Ex 1B - Axes + Main Line

Task:

In this exercise, you will implement the *draw* function inside the file *ex1_draw_line.py*.

To complete this task, carefully refer to the implementation of the *draw_line* function in *window_glut.py*.

Draw faint axes through the window center and one bold black line.



Hints:

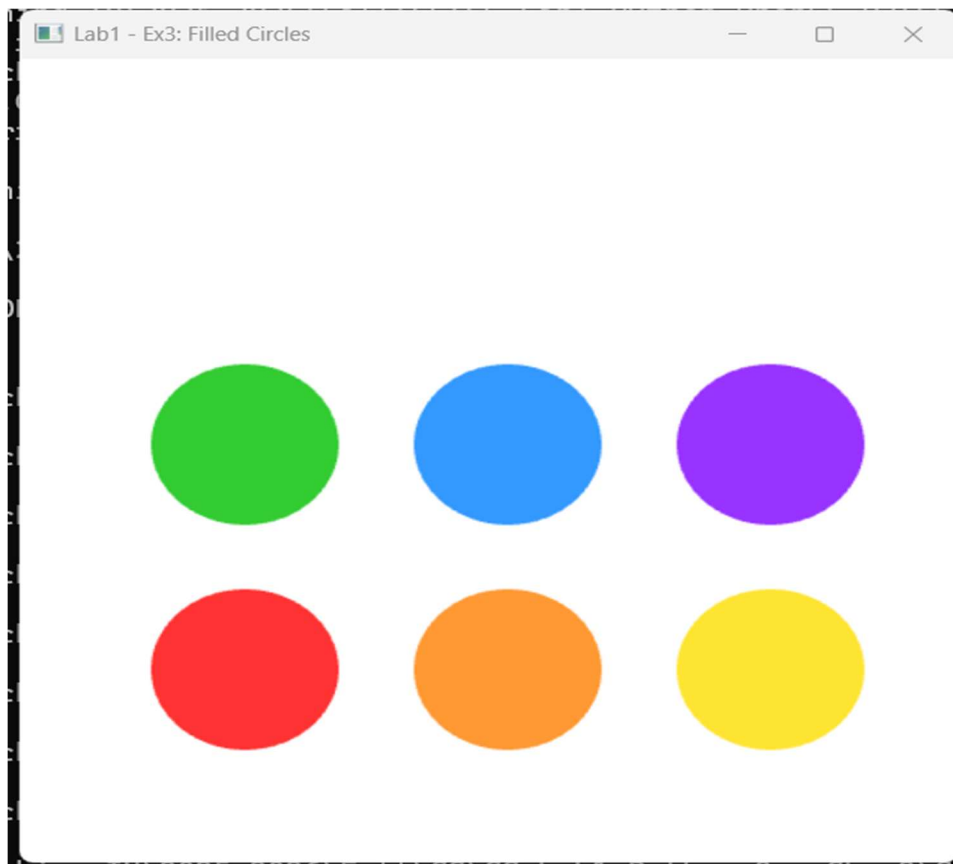
Center=(W/2, H/2); gray axes, thicker main line; avoid hard-coded sizes.

Ex 2A - Colored Filled Circles (Grid)**Task:**

In this exercise, you will implement the *draw* function inside the file *ex2_1_circles_fill.py*.

To complete this task, carefully refer to the implementation of the *draw_oval_filled* function in *window_glut.py*.

Draw a 3x2 grid of filled circles with different colors (optional black outlines).

**Hints:**

Nested loops; rectangle (cx-r, cy-r, 2r, 2r); cycle color list.

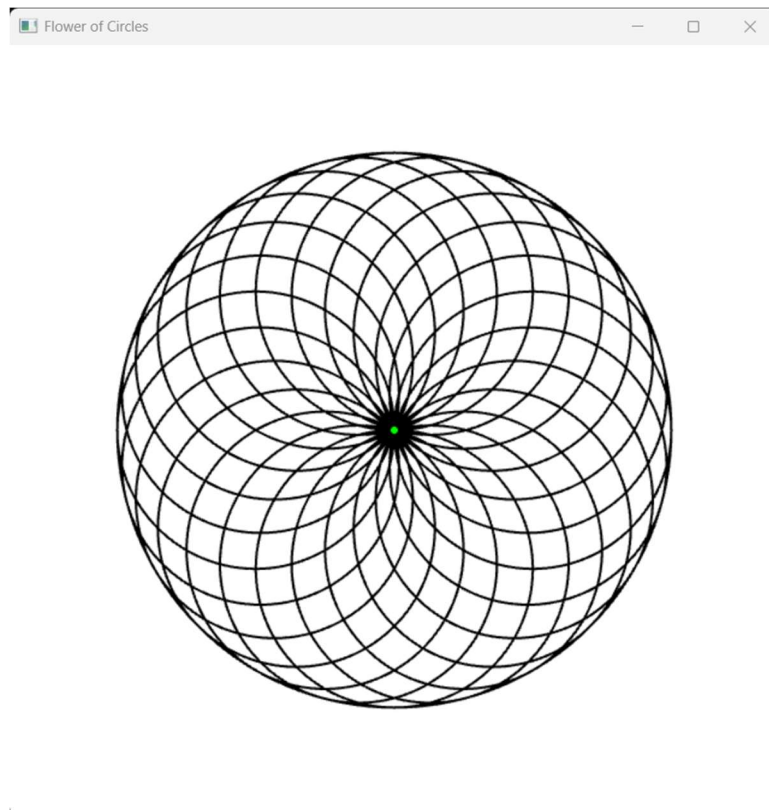
Ex 2B - Flower of Circles

Task:

In this exercise, you will implement the *draw* function inside the file *ex2_2_set_circles.py*.

To complete this task, carefully refer to the implementation of the *draw_oval_filled* and *draw_oval_outline* functions in *window_glut.py*.

Draw N equal circles with centers on radius R ; set $r=R$ so all pass through the center; add a big boundary and a small green dot.



Hints:

Center $(cx,cy)=(W/2,H/2)$; $\theta=2*\pi*k/N$; outline each circle.

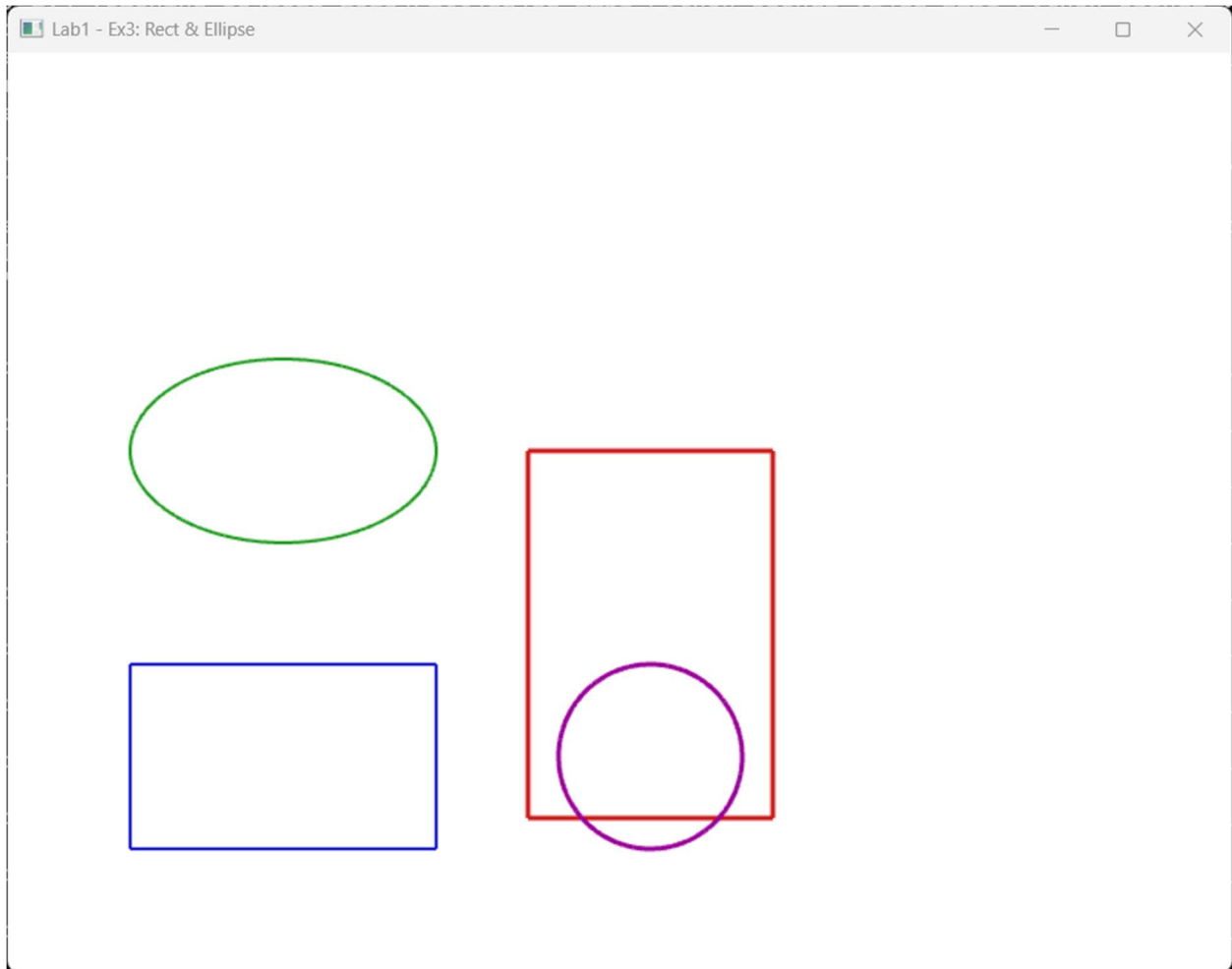
Ex 3 - Rectangles & Ellipses (Outlines)

Task:

In this exercise, you will implement the *draw* function inside the file *ex3_1_rect_ellipse.py*.

To complete this task, carefully refer to the implementation of the *draw_oval_outline* and *draw_rect_outline* functions in *window_glut.py*. You can modify the code to decorate each shape.

Draw at least two rectangles and two ellipses with different sizes and stroke widths.



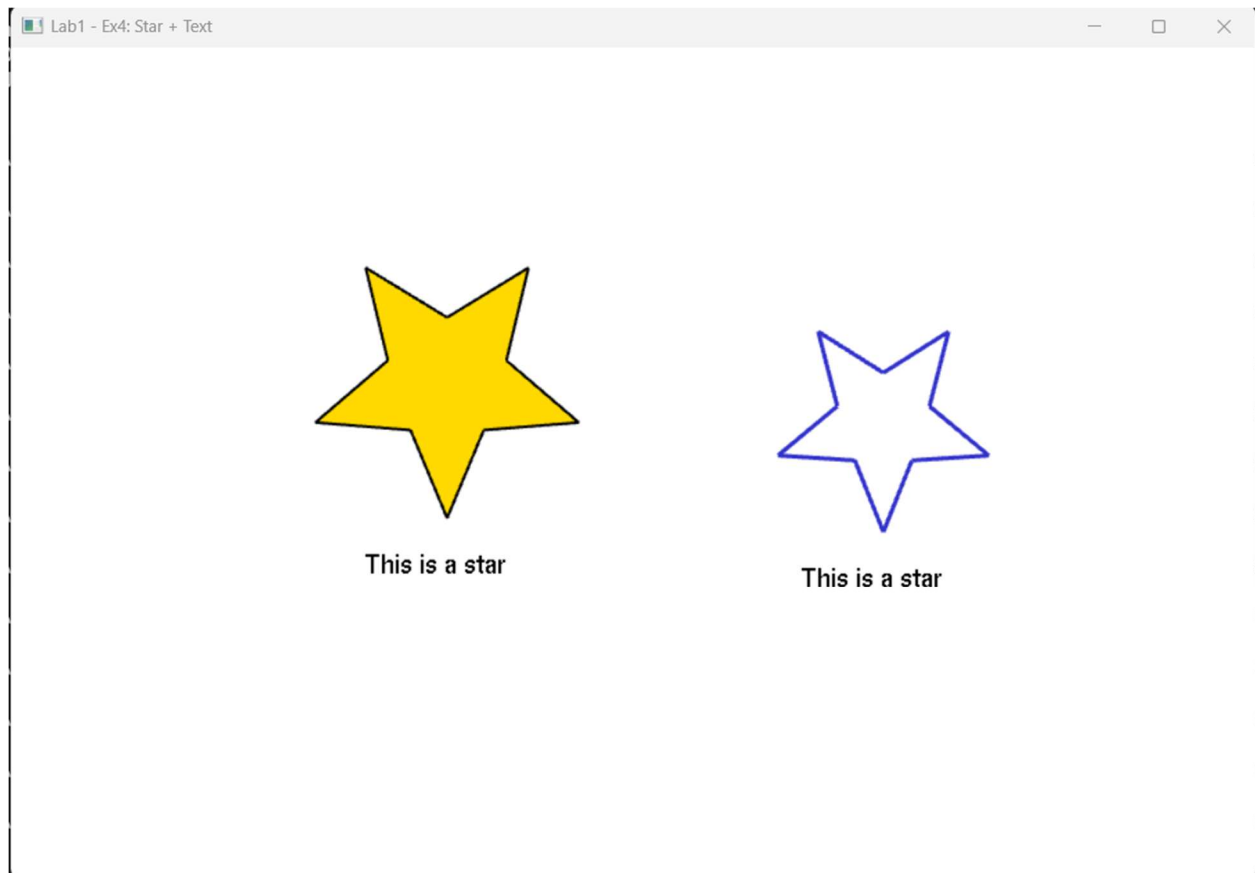
Ex 4A - Star + Caption

Task:

In this exercise, you will implement the *star_points* function and the *draw* function of the *App* class inside the file *ex4_star_and_text.py*.

To complete this task, carefully refer to the implementation of the *draw_polyline* and *draw_text* functions in *window_glut.py*.

Compute 10 vertices of a five-point star; fill one star and draw an outline-only star; add caption: This is a star.



Hints:

Outer at $-90+72*i$ deg; inner at $+36$ deg; fill with `GL_TRIANGLE_FAN`; outline with closed polyline.

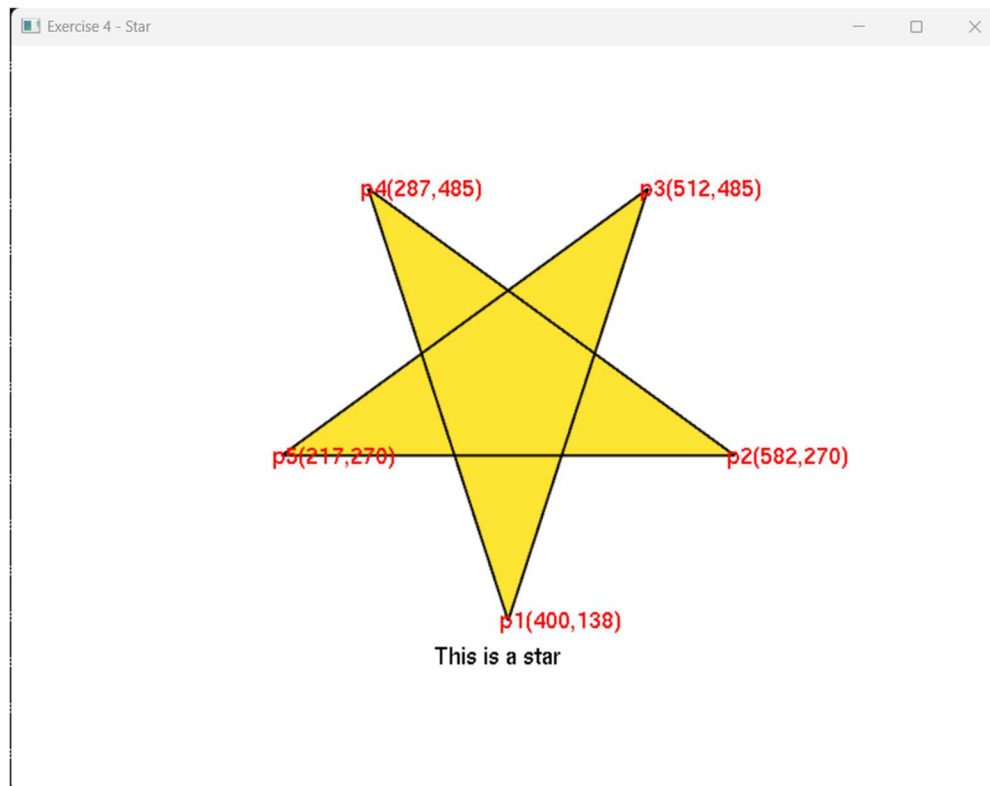
Ex 4B - Pentagram (Complete) + Labels

Task:

In this exercise, you will implement the *star_points* function and the *draw* function of the *App* class inside the file *ex4_star_complete.py*.

To complete this task, carefully refer to the implementation of the *draw_polyline* and *draw_text* functions in *window_glut.py*.

Render a pentagram with correct inner radius; fill; connect outer indices 0,2,4,1,3,0; label p1..p5 and add caption.



Hints:

Inner radius $r=R*\sin(18)/\sin(54)$; build alternating ring; triangle fan for fill; text near outer tips.

1. Compute outer and inner points with trigonometry (outer every 72° , inner shifted 36°).
2. Use `GL_TRIANGLE_FAN` from the center with alternating outer/inner points to fill the star.
3. Draw the pentagram outline by connecting every 2nd outer vertex ($0 \rightarrow 2 \rightarrow 4 \rightarrow 1 \rightarrow 3 \rightarrow 0$).
4. Optionally, draw the full 10-point contour with `GL_LINE_LOOP`.
5. Use `self.draw_text()` to label outer points (p1..p5) and add a caption.
6. Refer to `window_glut.py` for examples of drawing lines and text.

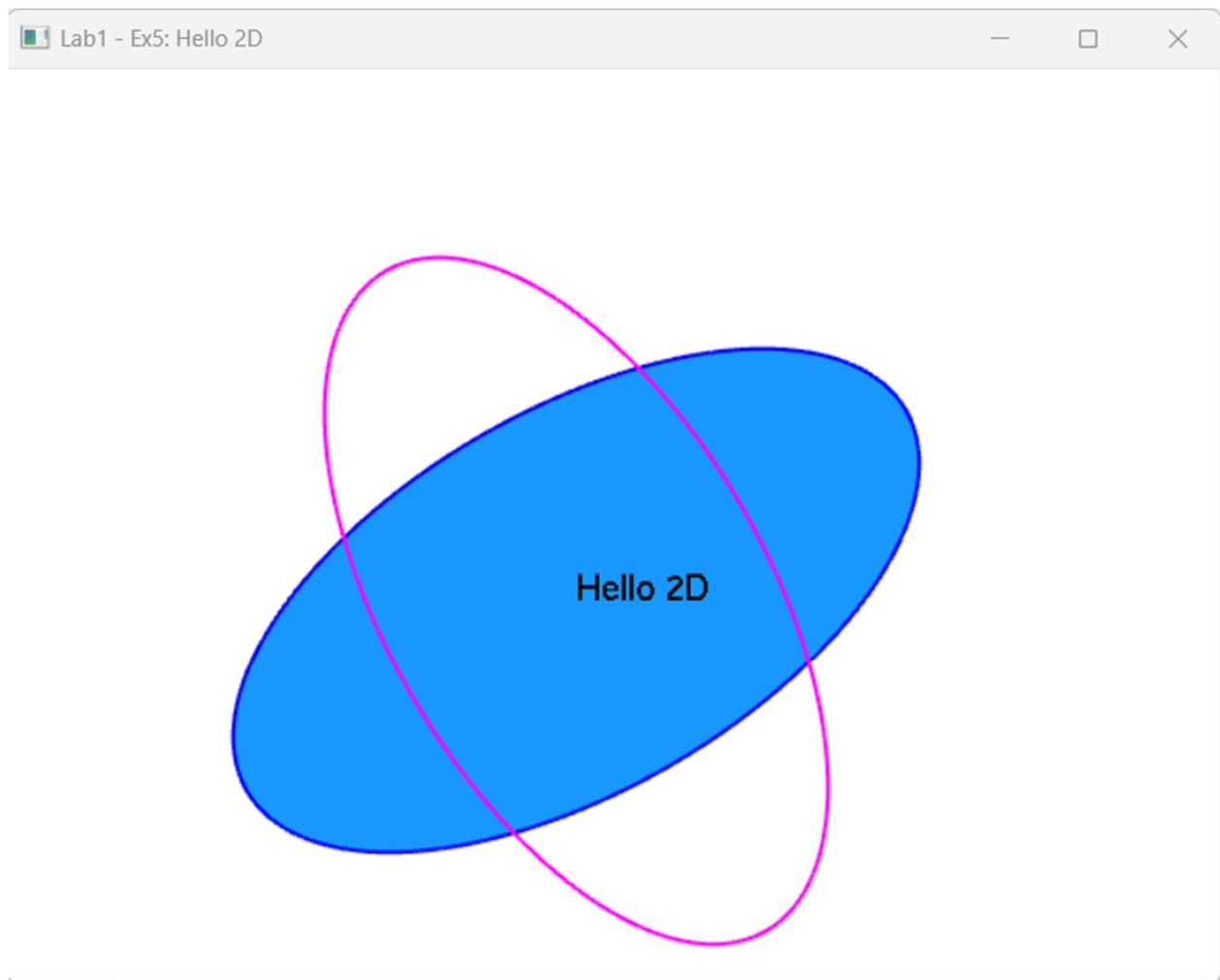
Ex 5 - Hello 2D Transforms + Filled Ellipses

Task:

In this exercise, you will implement the *draw_ellipse_filled* and *draw* functions of the *App* class inside the file *ex5_hello2d.py*.

To complete this task, carefully refer to the implementation of *window_glut.py*.

Apply `Translate(300,200) -> Scale(2,2) -> Rotate(30)`. Draw two ellipses with fill and outline; caption Hello 2D.



Hints:

Use push/pop; filled ellipse by `GL_TRIANGLE_FAN`; enable blending for translucent fills.

Part 2: Homework

Duration: 1 week

Materials: [Lab 1: Homework](#)

Reference:

https://www.youtube.com/watch?v=mOTE_62i5ag&list=PLn3eTxaOtL2PDnEVNwOgZFm5xYPr4dUoR&index=2

<https://www.youtube.com/watch?v=ZK1WyCMK12E&list=PLn3eTxaOtL2PDnEVNwOgZFm5xYPr4dUoR&index=4>

<https://www.youtube.com/watch?v=K9NeDMGHzfA&list=PLn3eTxaOtL2PDnEVNwOgZFm5xYPr4dUoR&index=5>

Exercise 1: Background Practice with PyOpenGL

You are given a simple program (window.py) that opens an OpenGL window using Pygame. The code can run in two modes:

Mode 1: Solid background color (set with `glClearColor`).

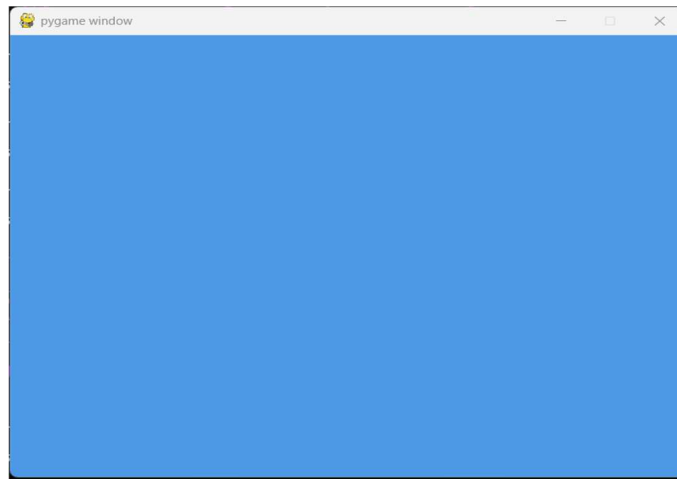
Mode 2: Image as background (loaded as an OpenGL texture).

Your Tasks

Part 1 – Background Color

Modify the code in Mode 1.

Change the values of `glClearColor(r,g,b,1)` to experiment with different background colors (red, green, blue, gray, etc.).

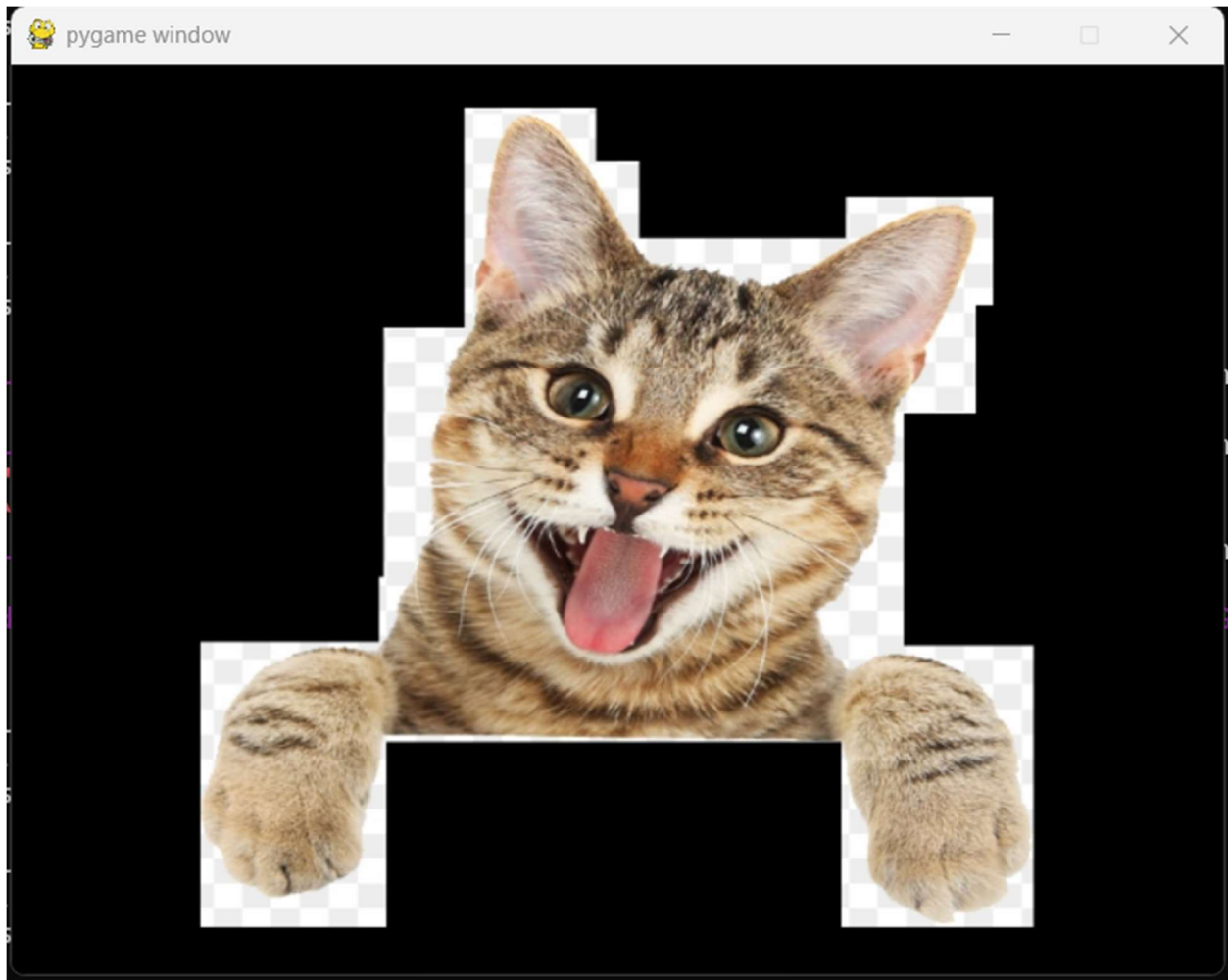


Part 2 – Background Image

Switch the code to Mode 2.

Replace `background.jpg` with your image and run the program.

(Optional) Try changing the texture coordinates in `_draw_background()` to see how the image is distorted.



Exercise 2: Introduction to Shaders with PyOpenGL + Pygame

You will learn how to use shaders with PyOpenGL and Pygame. Shaders are small GPU programs written in GLSL (OpenGL Shading Language) that control how vertices and pixels are processed. You will practice with a simple triangle rendered using a vertex shader and a fragment shader.

Provided Files

- triangle.py: Main Python program that initializes Pygame and OpenGL, loads shaders, and renders a triangle.
- shaders/vertex.txt: Vertex shader code.
- shaders/fragment.txt: Fragment shader code.

Vertex Shader (vertex.txt)

```
#version 330 core

layout (location=0) in vec3 vertexPos;
layout (location=1) in vec3 vertexColor;

out vec3 fragmentColor;

void main()
{
    gl_Position = vec4(vertexPos, 1.0);

    fragmentColor = vertexColor;
}
```

Fragment Shader (fragment.txt)

```
#version 330 core

in vec3 fragmentColor;

out vec4 color;

void main()
{
    color = vec4(fragmentColor, 1.0);
}
```

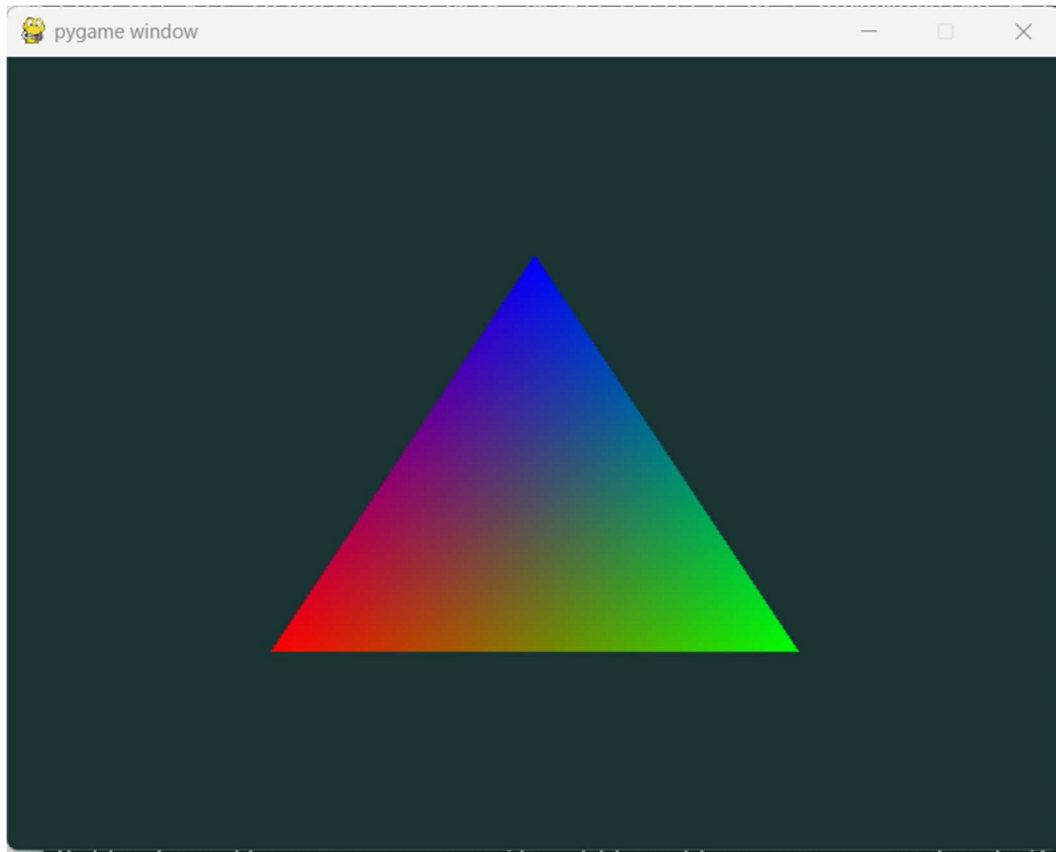
Running the Program

1. Place triangle.py and the shaders folder in the same directory.

2. Run the program with:

```
python triangle.py
```

3. A window should appear displaying a triangle with RGB-colored vertices.



Your tasks

Task 1: Change Vertex Colors

- Modify the vertex colors in `triangle.py`.
- Try different combinations such as yellow, cyan, or white.

Task 2: Modify the Fragment Shader

- In `fragment.txt`, make the triangle always red:
`color = vec4(1.0, 0.0, 0.0, 1.0);`
- Experiment with other solid colors.

Task 3: Apply Transformations in the Vertex Shader

- Scale the triangle horizontally by modifying `gl_Position`:
`gl_Position = vec4(vertexPos.x * 0.5, vertexPos.y, vertexPos.z, 1.0);`

Further Learning Resources

- OpenGL Shading Language (GLSL) Tutorial: <https://learnopengl.com/Getting-started/Hello-Triangle>
- PyOpenGL Documentation: <http://pyopengl.sourceforge.net/documentation/manual-3.0/>
- Pygame Documentation: <https://www.pygame.org/docs/>

Exercise 3: Textured Triangle with Shaders (PyOpenGL + Pygame)

In this Exercise, you will extend your knowledge of shaders by rendering a triangle with a texture. Instead of only coloring the triangle by vertex colors, we'll map an image (like `cat.png` or `wood.jpeg`) onto its surface.

Provided Files

`textured_triangle.py` → Main Python program (sets up OpenGL, loads shaders, triangle geometry, and texture).

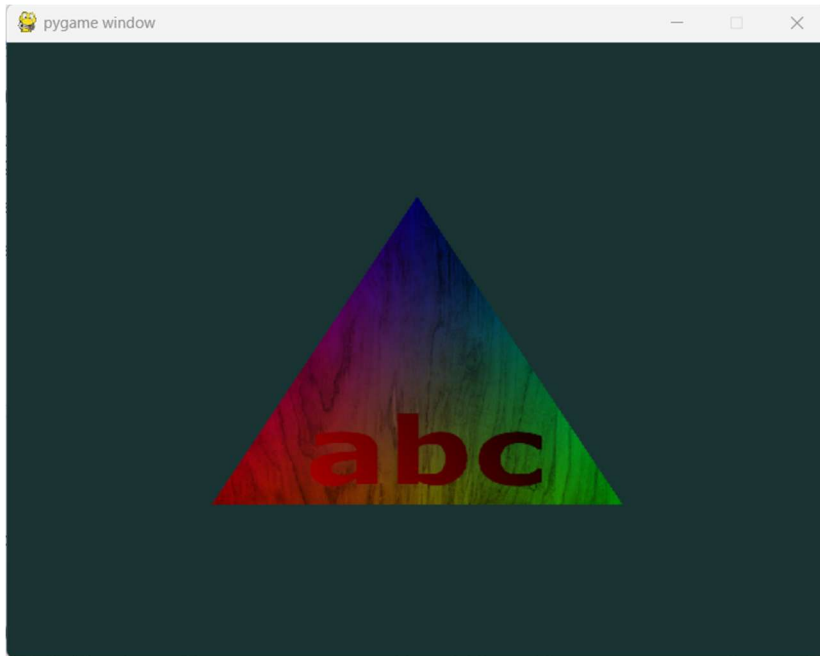
`shaders/vertex.txt` → Vertex shader with position, color, and texture coordinates

`shaders/fragment.txt` → Fragment shader that multiplies vertex color with the texture

`cat.png`, `wood.jpeg` → Example texture images.

Running the Program

Place `textured_triangle.py`, the shaders folder, and texture images (`cat.png`, `wood.jpeg`) in the same directory.



Your Tasks:

Task 1: Change the Texture

- Replace `cat.png` with `wood.jpeg` in `textured_triangle.py`.
- Observe the difference in how textures are mapped.

Task 2: Experiment with Texture Coordinates

- In the `Triangle` class, each vertex has (s, t) texture coordinates.
- Change them to flip the image horizontally or vertically.
- Example: set one coordinate to 0.0 instead of 1.0.

Task 3: Modify the Fragment Shader

- Ignore vertex colors:
`color = texture(imageTexture, fragmentTexCoord);`
- Ignore the texture:
`color = vec4(fragmentColor, 1.0);`
- Compare the results.

Task 4: Bonus Challenge

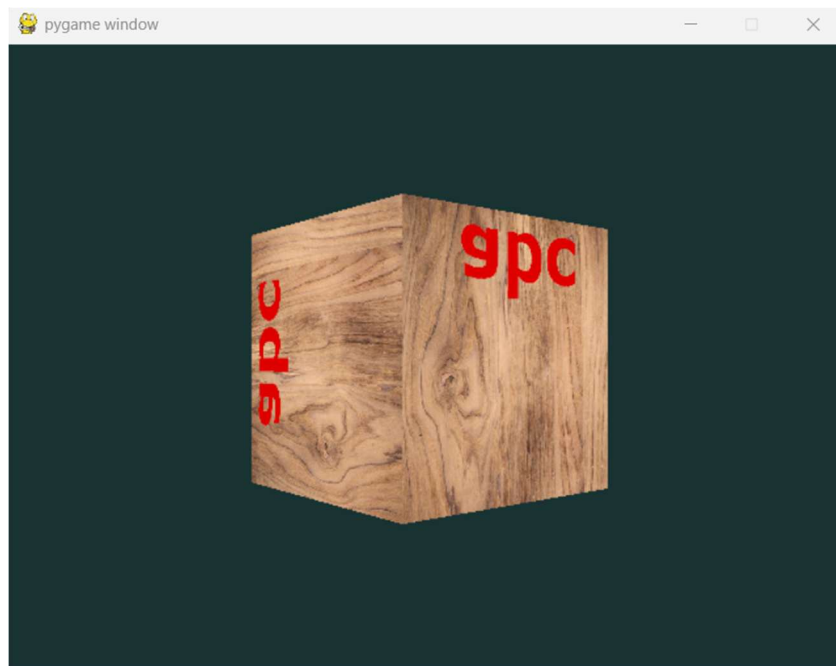
- Replace the image with your own (.png or .jpeg).
- Experiment with blending modes (e.g., add colors instead of multiply).

Exercise 4: Spinning Textured Cube (PyOpenGL + Pygame)

You will render a 3D spinning cube with a texture applied to its faces. You will learn how to use 3D transformations (projection and model matrices), apply textures to a 3D object, and continuously update the cube's rotation to make it spin.

Provided Files

- spinningCube.py: Main program (sets up shaders, cube mesh, and spinning entity).
 - shaders/vertex.txt: Vertex shader (handles projection + model transformations).
 - shaders/fragment.txt: Fragment shader (samples texture color).
 - wood.jpeg, cat.png: Example texture images.
1. Place spinningCube.py, the shaders folder, and texture images (wood.jpeg, cat.png) in the same directory.
 2. Run the program with:
python [spinningCube.py](#)
 3. A window should appear displaying a textured cube spinning around its Y-axis.



Your task

Task 1: Change the Texture

- Replace wood.jpeg with cat.png in spinningCube.py.
- Observe how the cube looks with different textures.

Task 2: Modify Rotation Speed

- In Entity.update(), the cube rotates by 0.25 each frame.
- Increase/decrease this value to change the rotation speed.

Task 3: Apply Different Transformations

- In Entity.get_model_transform(), change the rotation axis (X, Y, or Z).
- Add scaling to make the cube larger or smaller.

Task 4: Experiment with Shaders

- In the fragment shader, tint the texture:
`color = texture(imageTexture, fragmentTexCoord) * vec4(1.0, 0.5, 0.5, 1.0);`
- This adds a red tint. Try other color multipliers.

Task 5: Bonus Challenge

- Replace the cube with a pyramid or rectangular box.
- Apply a different image as a texture.